



**UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
INGENIERÍA EN REQUERIMIENTOS**

INFORME PARCIAL CORTE 2

ESTUDIANTE:

FRED ADRIÁN BELTRAN MONTIEL

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERON

CUARTO SEMESTRE A

Quevedo, 22 de febrero de 2026
SPA 2025-2026

1. Introducción

El presente informe tiene como finalidad analizar la integración de pruebas basadas en el enfoque Behavior Driven Development (BDD) utilizando la herramienta Cucumber y el lenguaje Gherkin en diferentes entornos de desarrollo. Se estudió su implementación en C# mediante SpecFlow (Reqnroll) y en Python mediante Behave, permitiendo comprender cómo los escenarios escritos en lenguaje natural pueden convertirse en pruebas automatizadas. A través de actividades prácticas, se exploró el proceso de instalación, configuración, creación de escenarios y ejecución de pruebas, evidenciando la importancia de la automatización en el aseguramiento de la calidad del software.

Semana 14 - Investigar el acoplamiento de Cucumber (lenguaje Gherkin) con la plataforma de C#

Cucumber en software

En desarrollo de software, **Cucumber** es una herramienta de *Behavior-Driven Development (BDD)* que permite escribir los requisitos y pruebas en lenguaje casi natural (Gherkin) y ejecutarlos como tests automáticos. Sirve como puente entre negocio y equipo técnico, mejorando comunicación, documentación y calidad [1]. Estudios recientes han demostrado que el uso de BDD con Cucumber facilita la colaboración entre equipos multidisciplinarios en proyectos de gran escala [6].

- Es un *framework* de pruebas BDD que interpreta especificaciones escritas en archivos **.feature** con pasos Given-When-Then y las enlaza con código de automatización (Java, Ruby, JavaScript, etc.) [2].
- Permite que usuarios de negocio describan el comportamiento esperado sin conocer el código, y los testers/desarrolladores implementan las "step definitions" que ejecutan esos escenarios.

Uso principal	Ejemplos típicos
Pruebas funcionales web / GUI	Testing de sitios (CURA, Swag Labs, e-commerce)
Aceptación de requisitos de negocio	Especificaciones ejecutables guiadas por historias de usuario
Integración en CI/CD	Integrado con Jenkins y pipelines de entrega continua
Otros dominios	Juegos 3D, servicios REST, simulación, BD relacionales

Cuadro 1: Usos de Cucumber en software

Beneficios clave

- **Mejor comunicación** entre negocio, testers y desarrolladores al compartir un lenguaje común Gherkin.
- **Documentación viva:** los **.feature** son a la vez requisitos y pruebas ejecutables, siempre actualizadas [3].
- **Mayor calidad y rapidez:** facilita automatizar regresión, smoke tests y pruebas de aceptación en ciclos ágiles/CI [7].

GHERKIN

Es un lenguaje estructurado, similar al lenguaje natural, usado en el desarrollo guiado por comportamiento (BDD) para describir características y escenarios de software mediante pasos *Given–When–Then* que sirven a la vez como especificación entendible por personas y como base para pruebas automatizadas [3]. Investigaciones empíricas han evaluado la calidad de los escenarios Gherkin, proponiendo marcos de trabajo para mejorar su claridad y mantenibilidad [9].

SpecFlow en software: herramienta BDD para .NET

En desarrollo de software, **SpecFlow** es una herramienta de *Behavior-Driven Development (BDD)* para el ecosistema **.NET**. Permite escribir requisitos y pruebas en lenguaje Gherkin (Given–When–Then) y ejecutarlos como tests automatizados, similar a Cucumber pero orientado a C# y .NET.

¿Qué es SpecFlow y cómo funciona?

- Es un *framework* BDD que toma escenarios escritos en Gherkin y los conecta con código de pruebas implementado en **C#/.NET** mediante "step definitions" [5].
- Su objetivo es **unir lenguaje de negocio y código**, reduciendo la brecha entre expertos de dominio, testers y desarrolladores.
- Se usa para pruebas de aceptación, funcionales y de regresión, donde los escenarios describen comportamientos del sistema en lenguaje casi natural [4].

Ejemplos de uso

Tipo de uso	Ejemplo breve
Pruebas de interfaces/plug-ins	Automatizar pruebas de un plugin de Excel
Pruebas de protocolos/infraestructura	Test automático de interfaz Modbus de un DCS
Ágil / Scrum con BDD	Mejorar colaboración dev-tester y cobertura

Cuadro 2: Ejemplos de uso de SpecFlow (Gherkin)

Beneficios y retos

- Mejora **colaboración y entendimiento de requisitos** al usar escenarios Gherkin compartidos por negocio, QA y desarrollo [8].
- Ayuda a crear **documentación viva**: los escenarios son a la vez especificación y prueba ejecutable.
- Retos frecuentes: curva de aprendizaje para escribir buenos escenarios y **mantenimiento** de los mismos a lo largo del tiempo [2].

1. INTEGRACIÓN CON C#

Se investigó el funcionamiento de Cucumber como herramienta de pruebas basadas en comportamiento (BDD - Behavior Driven Development), utilizando el lenguaje Gherkin para la definición de escenarios en formato estructurado (Given, When, Then).

En el entorno de C#, se identificó que Cucumber se implementa mediante la herramienta SpecFlow, la cual permite integrar archivos .feature escritos en Gherkin con proyectos desarrollados en .NET.

2. INSTALACIÓN

Se realizaron los siguientes pasos:

- Abrir la aplicación de **Visual Studio**.
- Creación de un proyecto de pruebas en C#.
- Instalación del paquete SpecFlow (Reqnroll) desde NuGet.
- Configuración del entorno para reconocer archivos .feature.

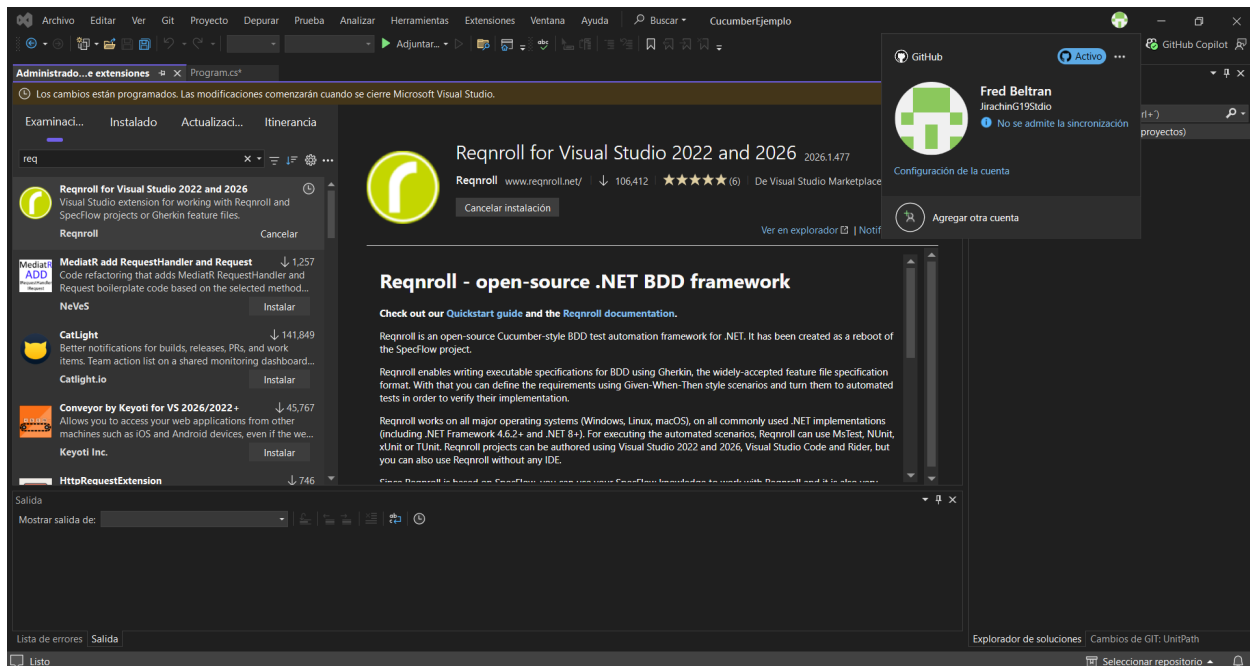


Figura 1: Instalación de Extension Reqnroll en proyecto C#

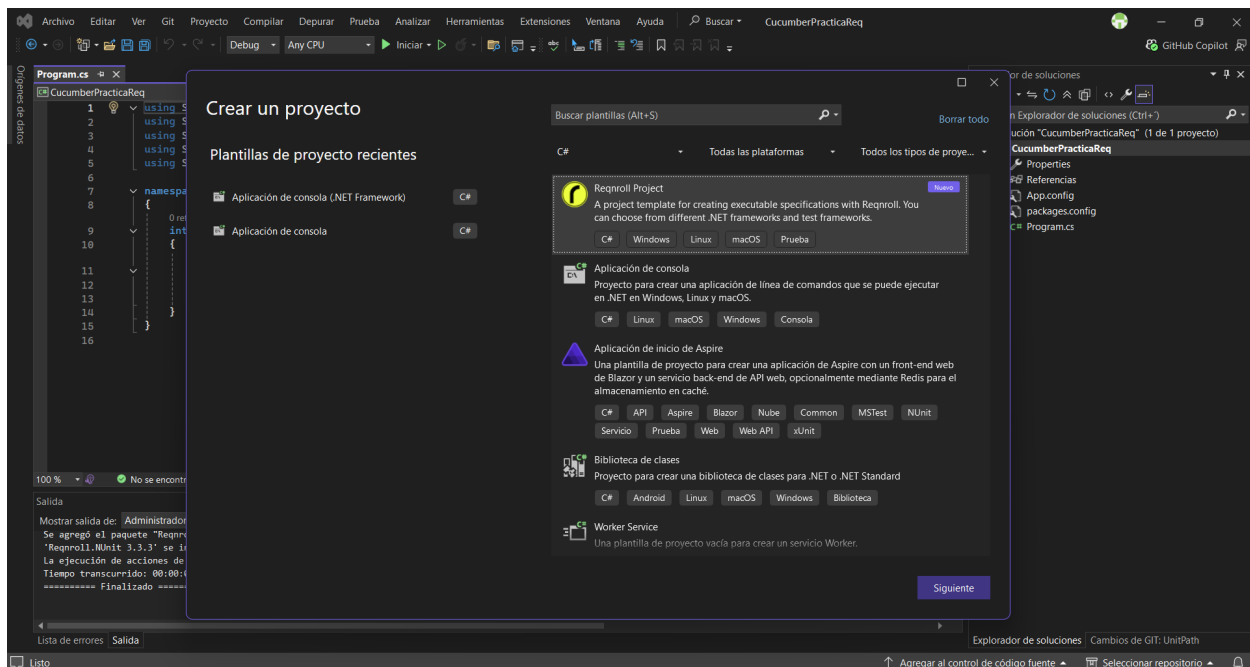


Figura 2: Instalación de paquetes NuGet de Reqnroll

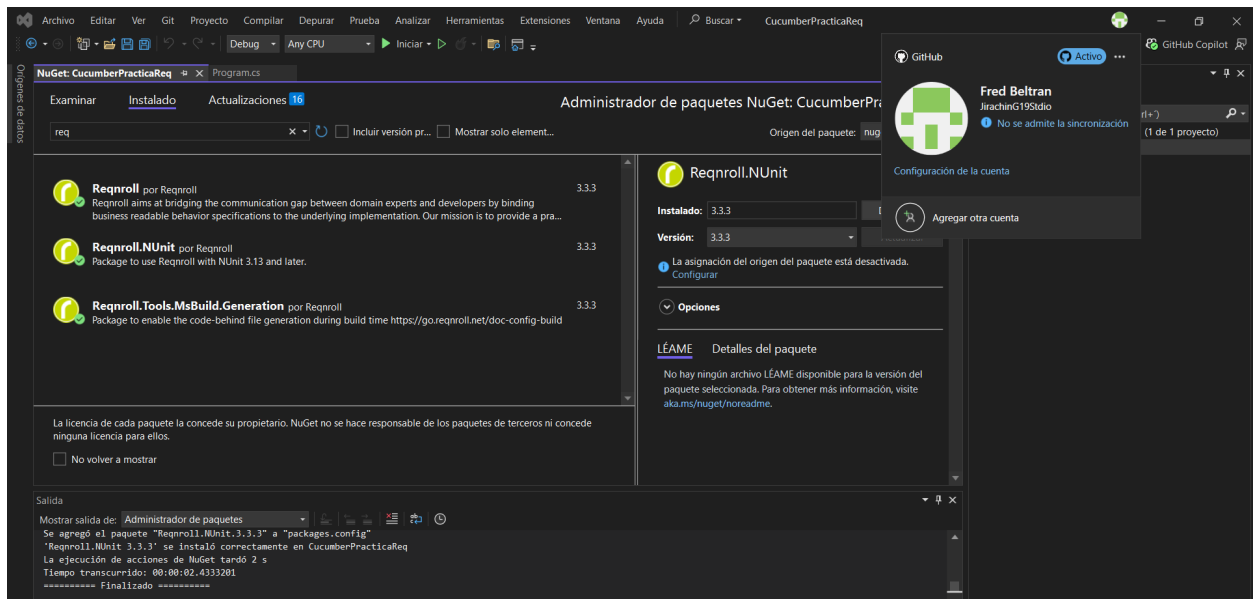


Figura 3: Creación del proyecto de pruebas (Reqnroll Project)

Agregar un archivo .feature

1. Clic derecho en el proyecto y Add
2. New Item
3. Selecciona: **Feature File**

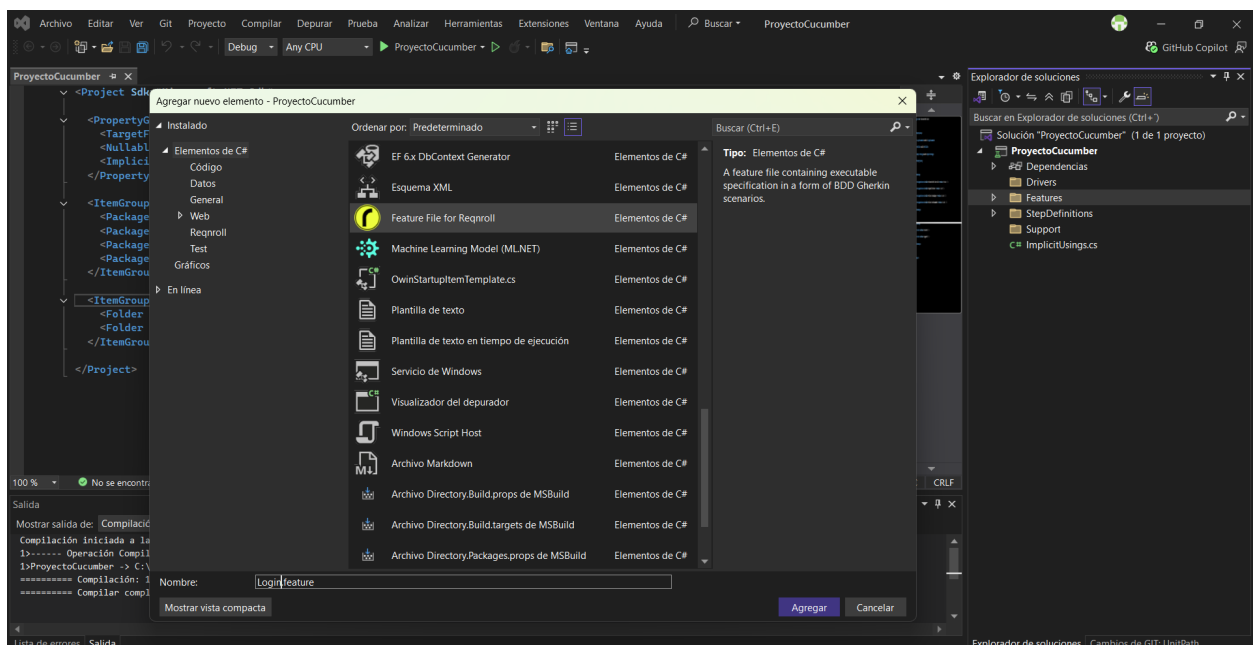


Figura 4: Creación de archivo .feature

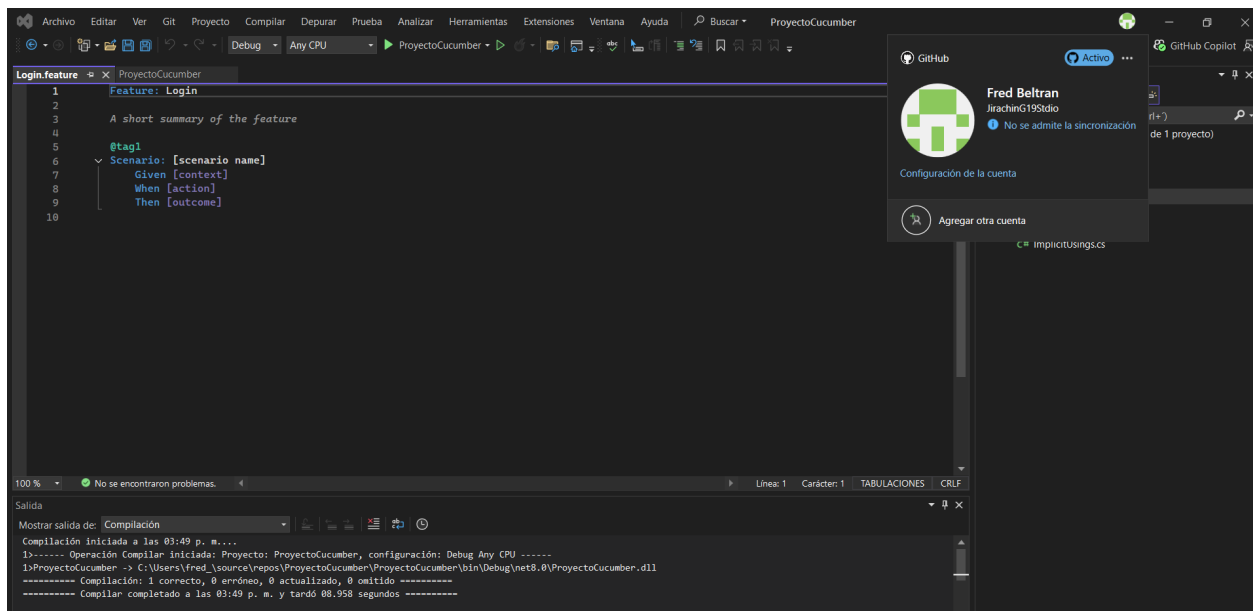


Figura 5: Muestra de .feature creada exitosamente

3. Escritura de Escenarios

Se creó un archivo .feature utilizando sintaxis Gherkin.

Ejemplo:

Feature: Login de usuario

Scenario: Inicio de sesión exitoso

Given que el usuario está en la página de login

When ingresa credenciales válidas

Then el sistema muestra el panel principal

Posteriormente se generaron los Step Definitions en C# para vincular cada paso con código ejecutable.

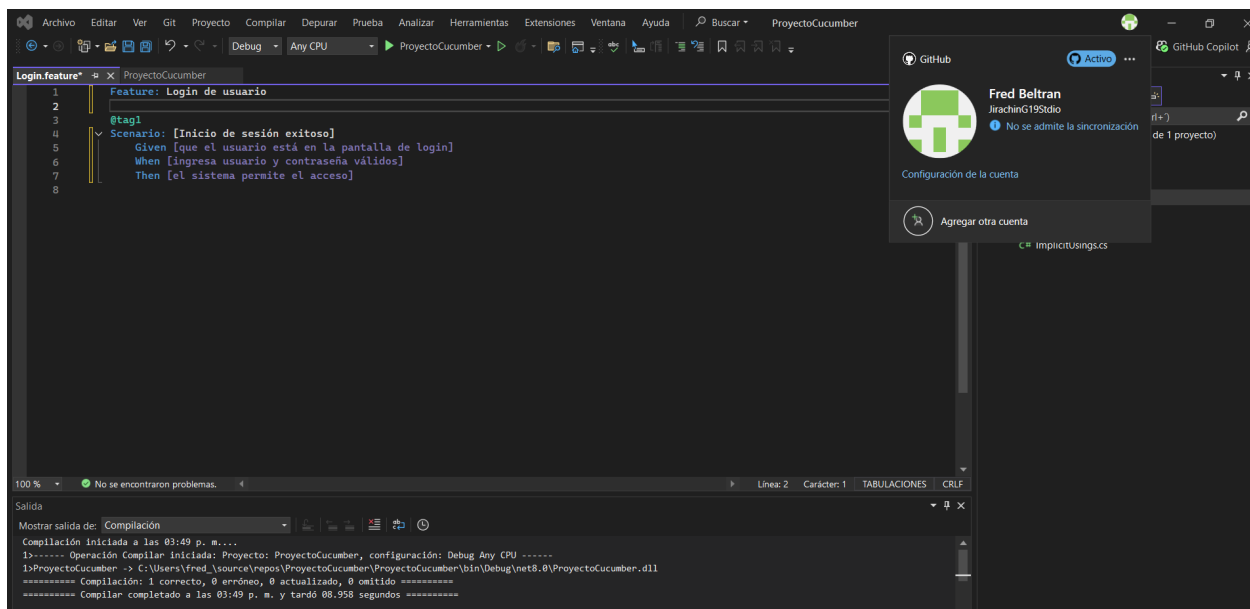


Figura 6: Escritura de escenarios

Generar Step Definitions

1. Guarda el archivo
2. Haz clic derecho dentro del escenario
3. Busca una opción como: "Define Steps" (incluye el icono de Reqrroll).

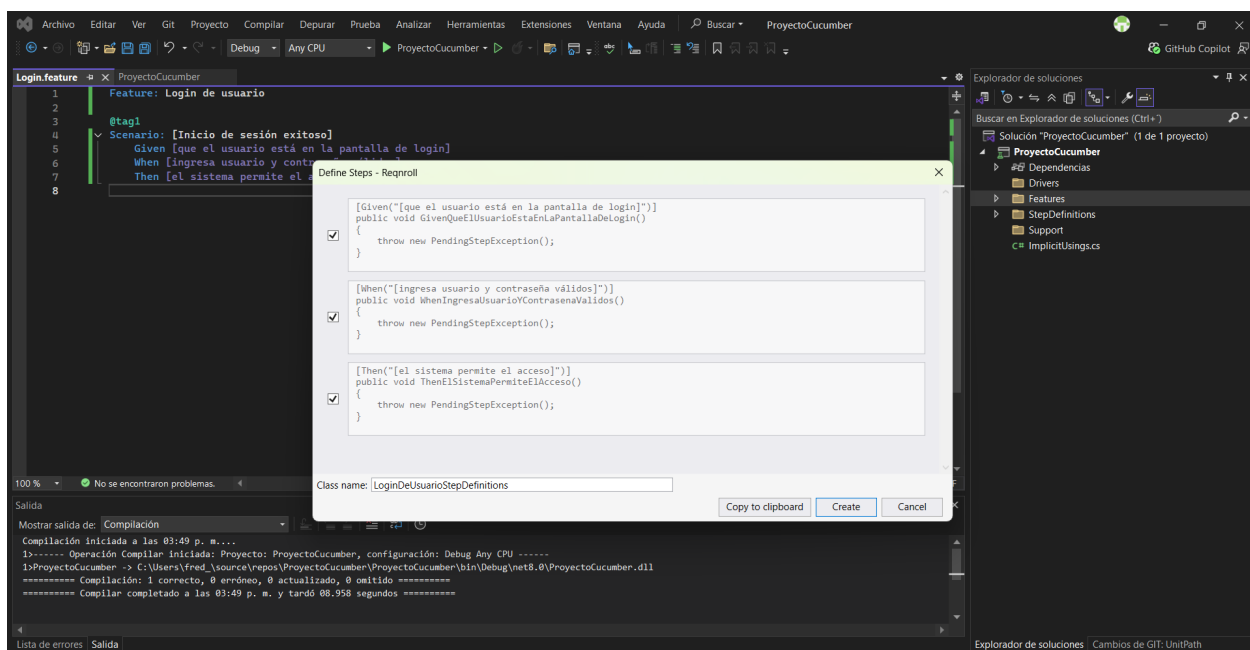


Figura 7: Generación de Step Definitions

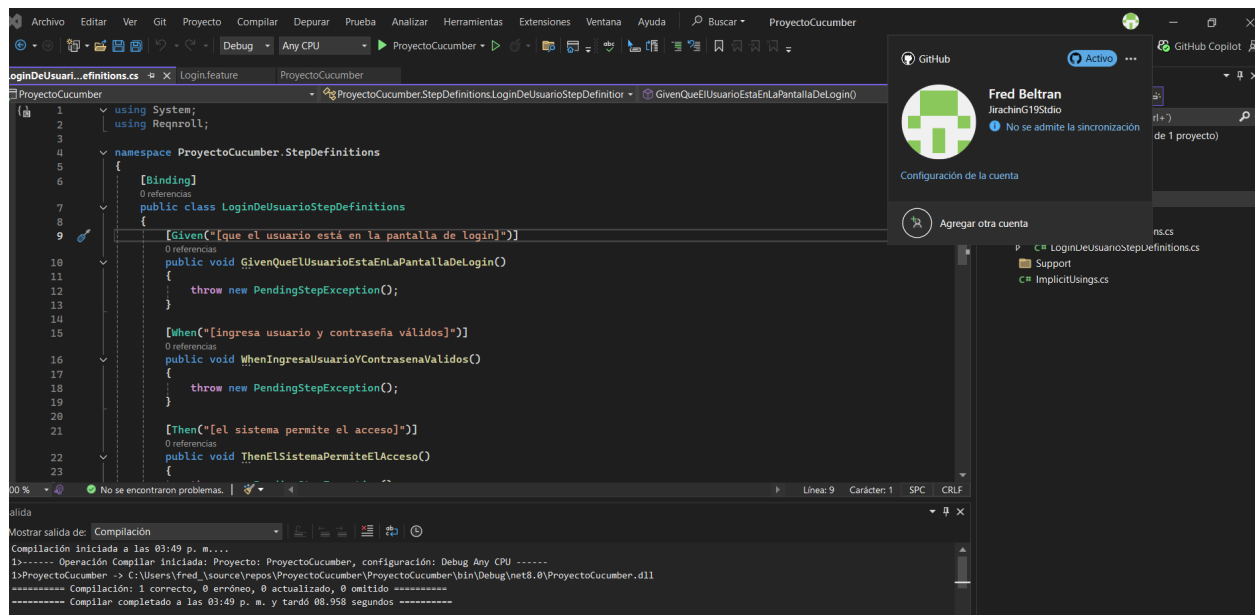


Figura 8: Implementación de los Define Steps

4. Prueba y Validación

Se ejecutaron los escenarios desde el Test Explorer de Visual Studio, verificando que:

- Los pasos fueran correctamente reconocidos.
- Las pruebas se ejecutaran sin errores.
- Los resultados fueran visibles en la consola de pruebas.

Se validó que el acoplamiento entre Gherkin y C# funciona correctamente mediante SpecFlow.

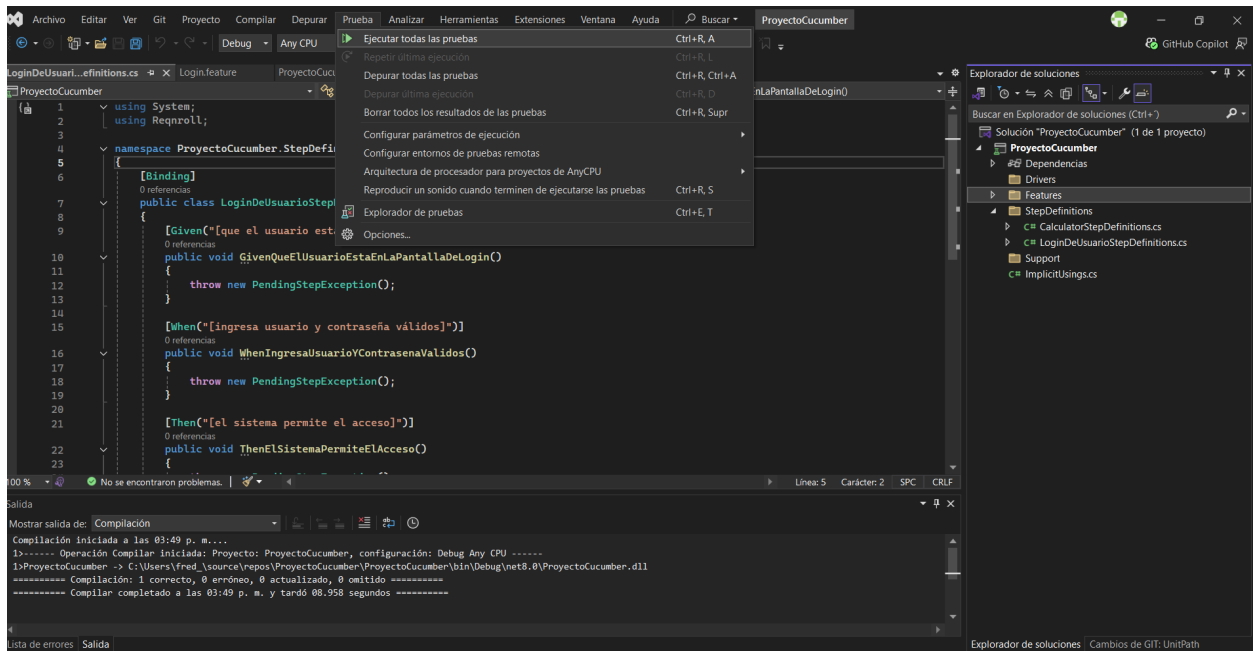


Figura 9: Seleccionamos ejecutar todas las pruebas

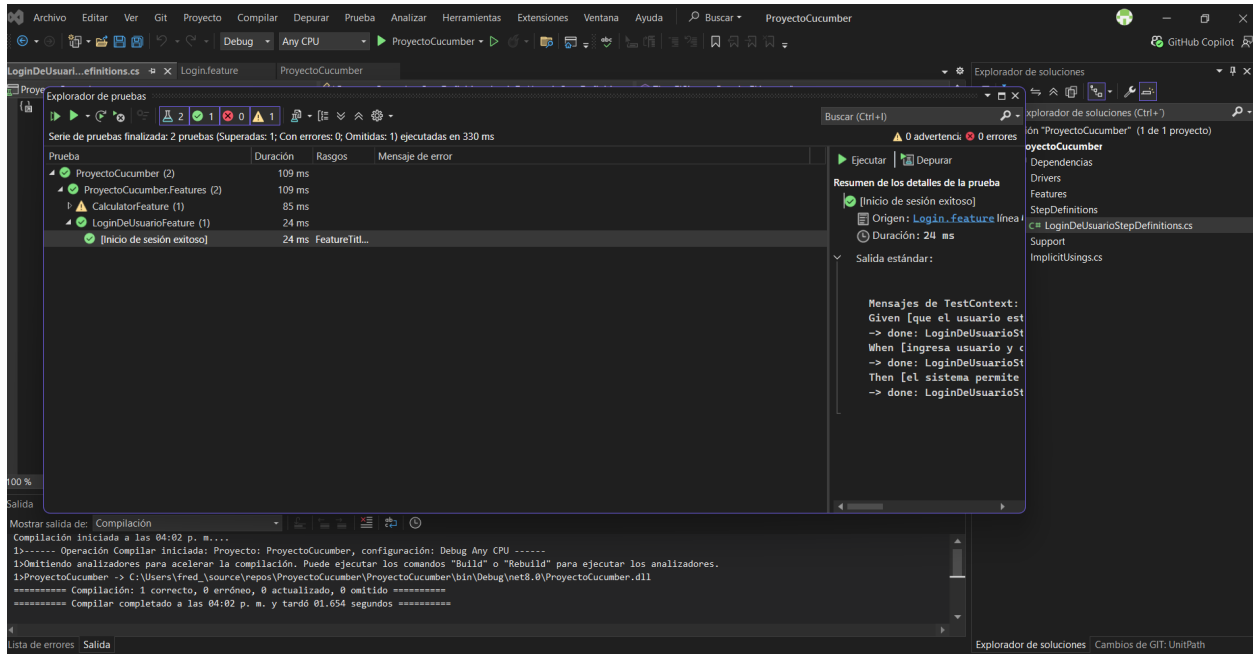


Figura 10: Ejecución exitosa de las pruebas

Semana 15 - Demostrar la automatización de pruebas en cucumber + Python

1. Instalación

Se instaló Python y la herramienta Cucumber para Python llamada Behave, que permite trabajar con Gherkin.

Instalar Python

1. Ve a la página oficial de Python
2. Descarga la versión estable.
3. Marcar la opción: "Add Python to PATH"

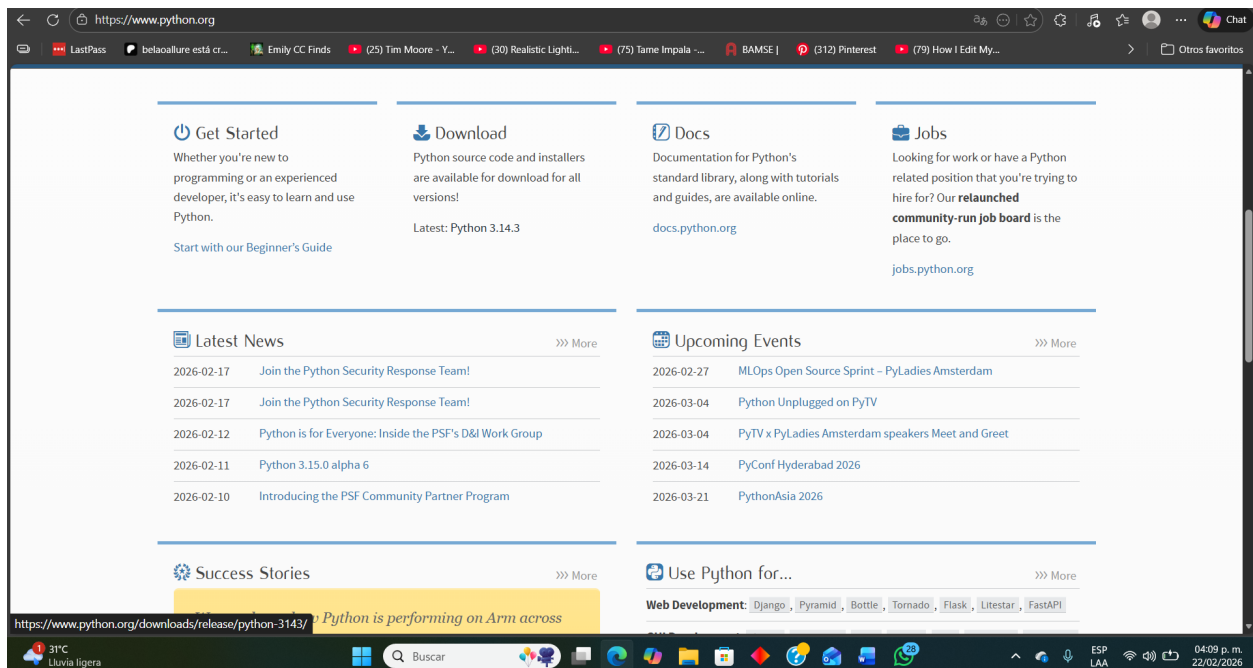


Figura 11: Instalación de Python en su Official Page

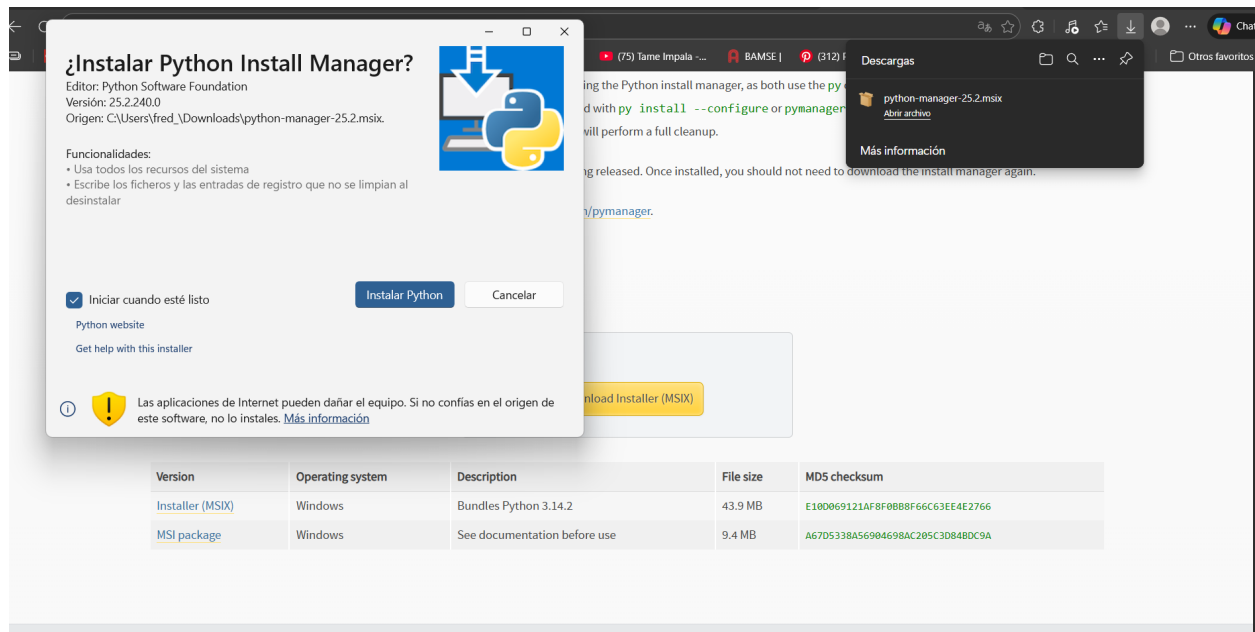


Figura 12: Instalación de Python

2. Configuración en Visual Studio Code

Se utilizó **Visual Studio Code** como entorno de desarrollo.
Configuraciones realizadas:

- Instalación de la extensión de Python.
- Instalación de extensión para soporte de Gherkin.
- Creación de estructura de carpetas:
 - features/
 - steps/

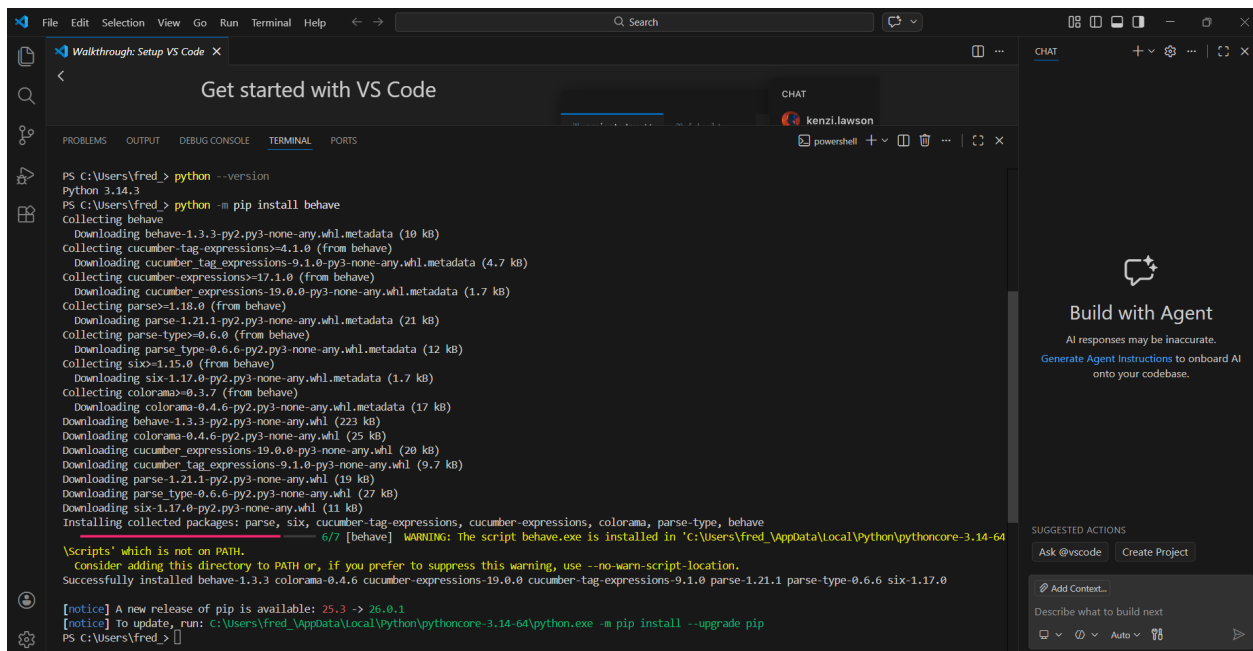


Figura 13: Instalación de las extensiones de python y behave en la terminal

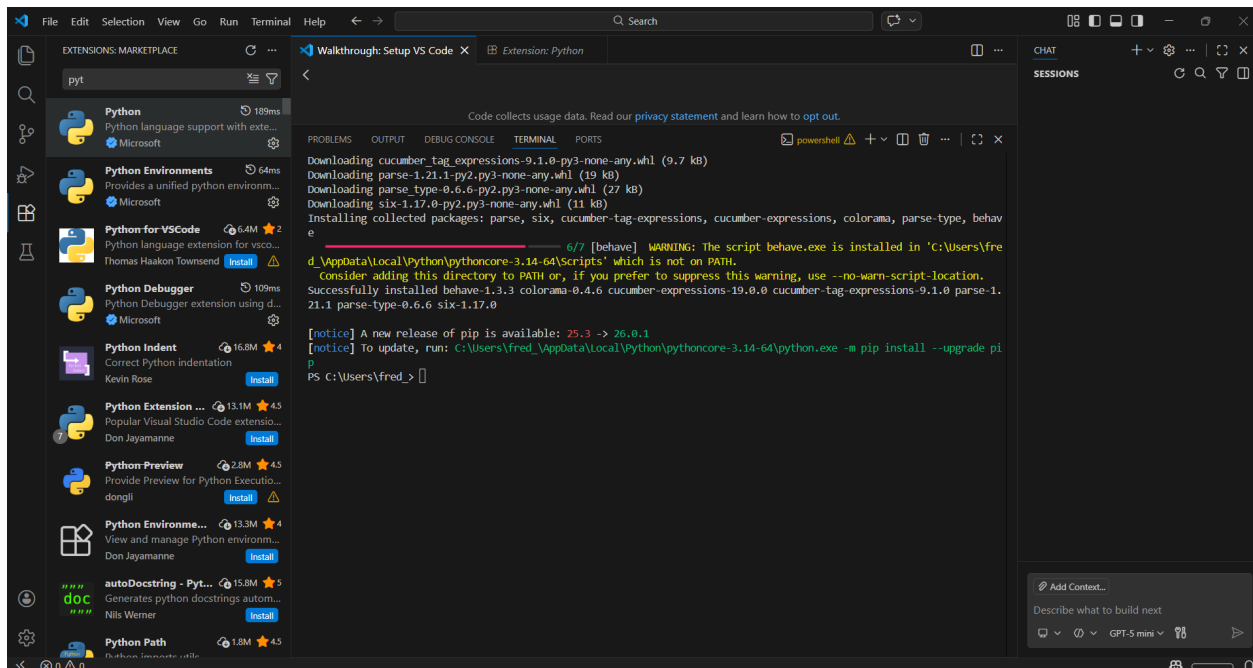


Figura 14: Instalación de las extensiones de Python

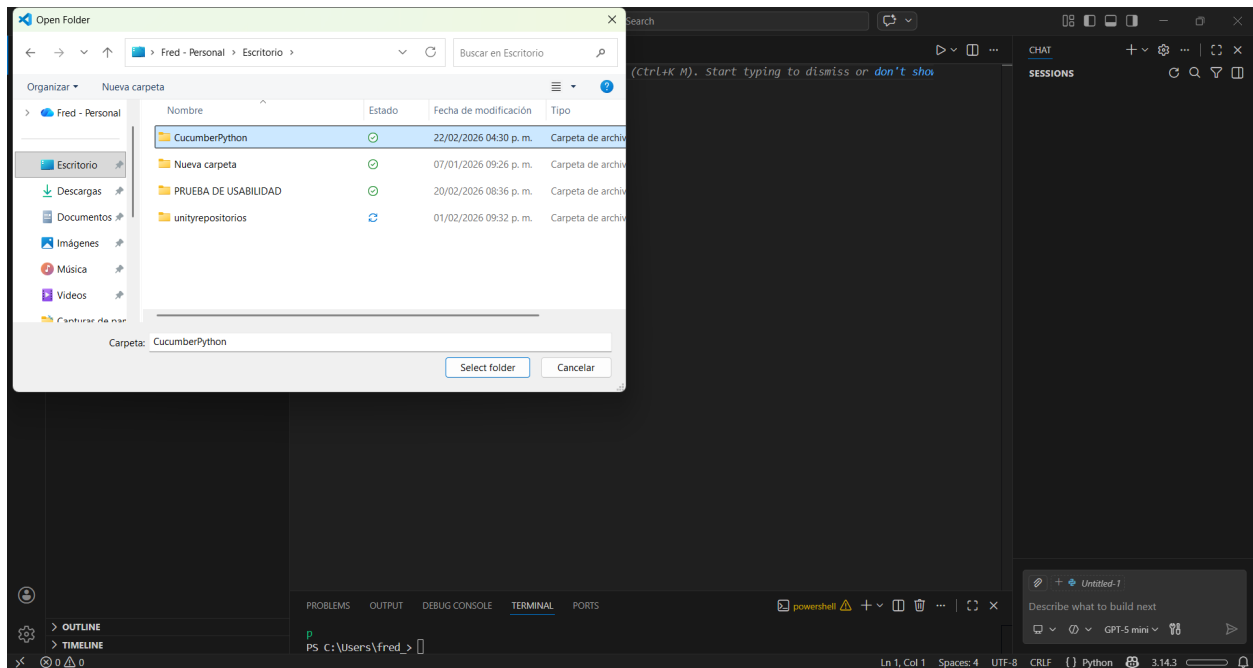


Figura 15: Crear y abrir un file para el proyecto

Crear una jerarquía para el entorno python

CarpetaCreada

features

login.feature

steps

login_steps.py

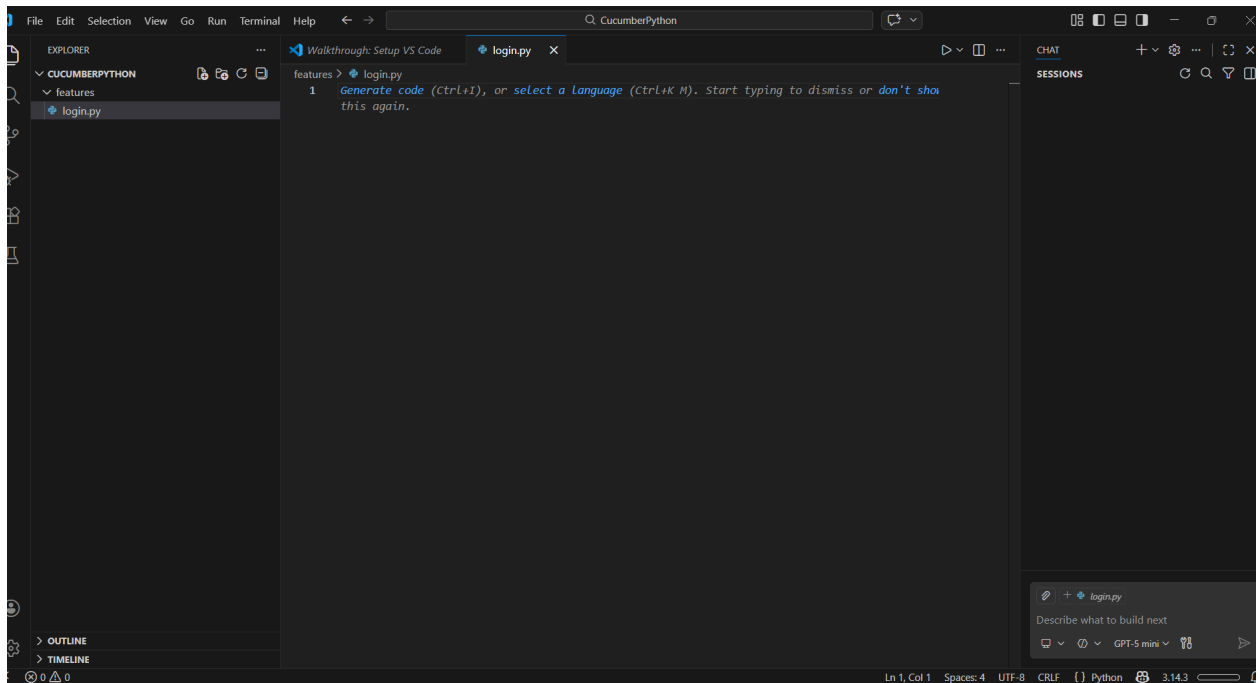


Figura 16: Crear una jerarquía de carpetas

3. Escritura de Escenarios

Se creó un archivo login.feature:

Feature: Login

Scenario: Usuario válido

Given el usuario ingresa usuario correcto

When ingresa contraseña correcta

Then el sistema permite el acceso

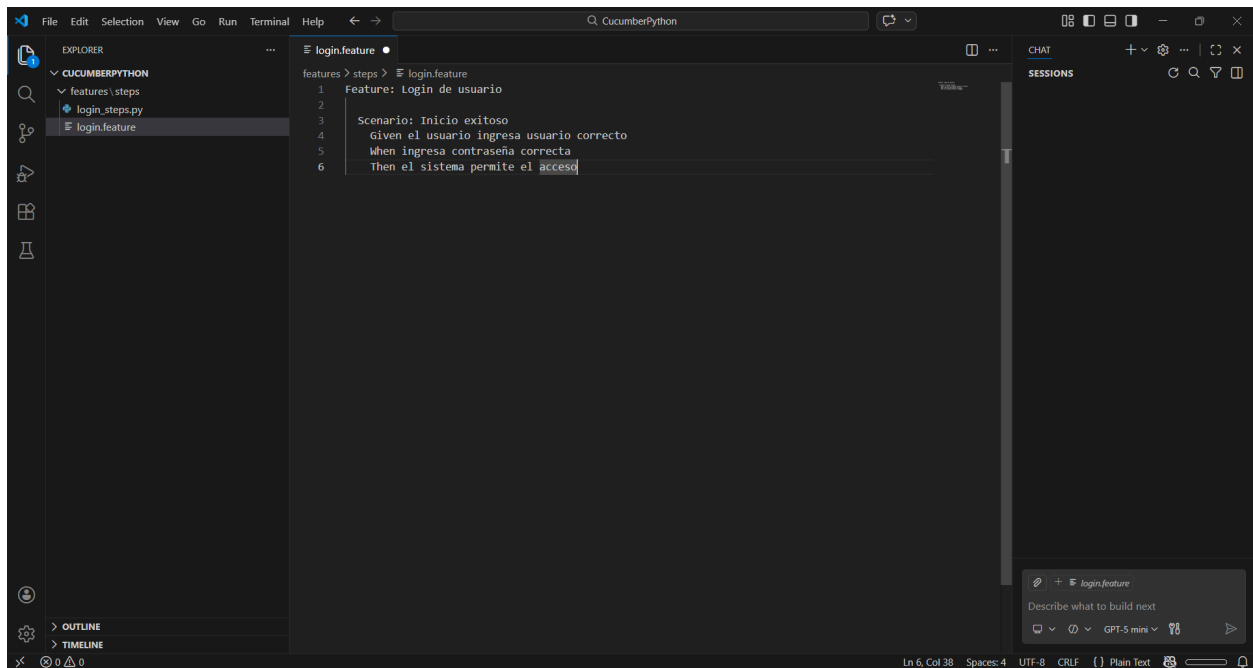


Figura 17: Escritura del escenario en python

4. Implementación de Steps en Python

En la carpeta steps se creó el archivo con las definiciones:

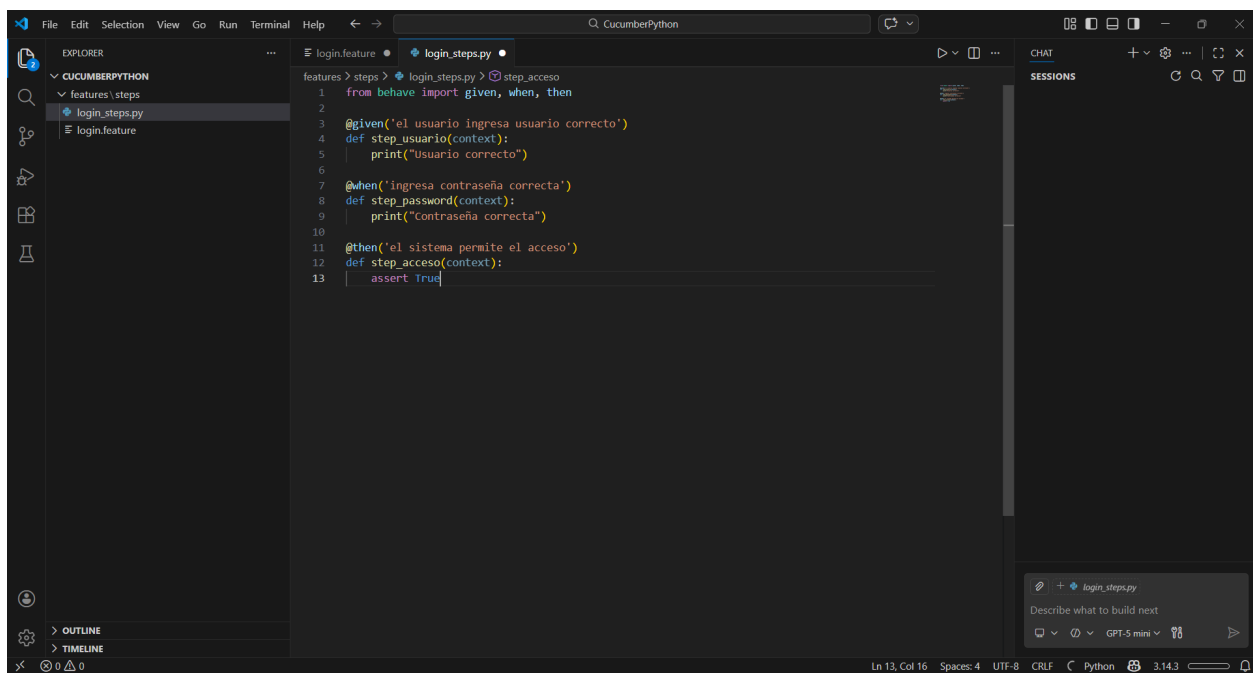


Figura 18: Implementación de los Steps en el archivo .py

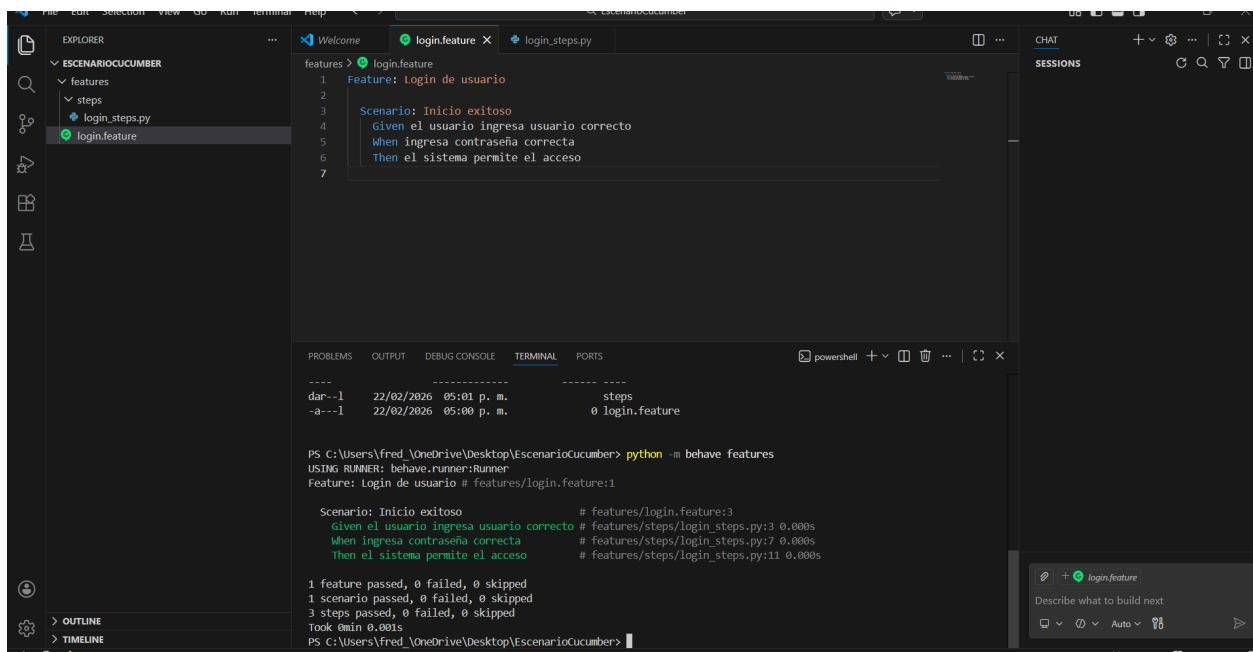
5. Ejecución del Escenario

Desde la terminal se ejecutó:

behave

Se verificó que:

- El escenario fue reconocido.
- Los pasos se ejecutaron correctamente.
- El resultado mostró la prueba como exitosa.



```
features > login.feature
1 Feature: Login de usuario
2
3 Scenario: Inicio exitoso
4   Given el usuario ingresa usuario correcto
5   When ingresa contraseña correcta
6   Then el sistema permite el acceso
7

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
-----
dar--1 22/02/2026 05:01 p. m. steps
-a--1 22/02/2026 05:00 p. m. 0 login.feature

PS C:\Users\Fred_OneDrive\Desktop\EscenarioCucumber> python -m behave features
USING RUNNER: behave.runner:Runner
Feature: Login de usuario # features/login.feature:1

Scenario: Inicio exitoso # features/login.feature:3
  Given el usuario ingresa usuario correcto # features/steps/login_steps.py:3 0.000s
  When ingresa contraseña correcta # features/steps/login_steps.py:7 0.000s
  Then el sistema permite el acceso # features/steps/login_steps.py:11 0.000s

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 0min 0.001s
PS C:\Users\Fred_OneDrive\Desktop\EscenarioCucumber>
```

Figura 19: Ejecución exitosa de la prueba

Conclusión

Durante las semanas 14 y 15 se logró comprender la integración de pruebas BDD utilizando Cucumber con dos entornos diferentes: C# mediante SpecFlow y Python mediante Behave. Se realizó la instalación, configuración, creación de escenarios en Gherkin y ejecución de pruebas automatizadas. Esto permitió reforzar el conocimiento sobre pruebas automatizadas y la importancia del desarrollo guiado por comportamiento en proyectos de software. En conclusión, el estudio realizado permitió comprender de manera práctica el acoplamiento de Cucumber con distintos entornos tecnológicos, específicamente C a través de SpecFlow (Reqnroll) y Python mediante Behave. La implementación de escenarios en lenguaje Gherkin demostró cómo el enfoque BDD facilita la comunicación entre los actores del proyecto y transforma los requisitos en pruebas ejecutables, fortaleciendo la calidad del producto final. Además, la experiencia práctica evidenció que la automatización de pruebas no solo optimiza el tiempo de validación, sino que también reduce errores humanos y mejora la trazabilidad de los requisitos. En conjunto, estas herramientas representan una estrategia efectiva para proyectos ágiles, promoviendo una documentación viva y una mayor alineación entre negocio y desarrollo.

Referencias

- [1] Y. Wang, L. Jia, H. Cao, Z. Jing and H. Huang, "Applications of Cucumber on Automated Functional Simulation Testing," *2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, 2021, pp. 861-862, doi: 10.1109/QRS-C55045.2021.00129.
- [2] S. Srinivas and L. Goel, "Designing and Implementing Robust Test Automation Frameworks using Cucumber BDD and Java," *arXiv preprint arXiv:2505.17168*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.17168>
- [3] A. Chandorkar, N. Patkar, A. Di Sorbo and O. Nierstrasz, "An Exploratory Study on the Usage of Gherkin Features in Open-Source Projects," *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2022, pp. 1159-1166, doi: 10.1109/SANER53432.2022.00134.
- [4] P. K. Koppanati, "Behavior-Driven Development (BDD) for Insurance Domain Testing," *International Journal For Multidisciplinary Research*, vol. 2, no. 6, 2020, doi: 10.36948/ijfmr.2020.v02i06.6132.
- [5] C. Wang, F. Pastore, A. Goknil and L. C. Briand, "Automatic Generation of Acceptance Test Cases From Use Case Specifications: An NLP-Based Approach," *IEEE Transactions on Software Engineering*, vol. 48, no. 2, pp. 585-616, 1 Feb. 2022, doi: 10.1109/TSE.2020.2998503.
- [6] M. S. Farooq, U. Omer, A. Ramzan, M. A. Rasheed and Z. Atal, "Behavior Driven Development: A Systematic Literature Review," *IEEE Access*, vol. 11, pp. 88008-88024, 2023, doi: 10.1109/ACCESS.2023.3302356.
- [7] M. Irshad, R. Britto and K. Petersen, "Adapting Behavior Driven Development (BDD) for large-scale software systems," *Journal of Systems and Software*, vol. 177, 110944, 2021, doi: 10.1016/j.jss.2021.110944.
- [8] J. Shi, J. Mönnich, J. Klünder and K. Schneider, "Organizing Graphical User Interface tests from behavior-driven development as videos to obtain stakeholders' feedback," *Journal of Software: Evolution and Process*, vol. 36, no. 12, e2721, 2024, doi: 10.1002/smr.2721.
- [9] D. W. Sears, K. Tsilionis and Y. Wautelet, "Improving Behavior-Driven Development Scenarios: Empirical Evaluation of a Quality Assessment Framework," in *Product-Focused Software Process Improvement: 26th International Conference, PROFES 2025*, Lecture Notes in Computer Science, vol. 16361, Springer, 2025, pp. 303-318, doi: 10.1007/978-3-032-12089-2_19.